

Лабораторная работа № 5

Установка PostgreSQL и подключение Prisma ORM

1. Место работы в курсе

В лабораторных работах № 1–4 студент создал приложение StudySpace на Next.js, настроил App Router, разработал клиентскую галерею учебных мест, модальное окно, бургер-меню, карту и адаптивные стили. Данные пока находятся в TypeScript-массиве, а состояние пользователя и избранного хранится только на клиенте.

Пятая работа начинает серверный блок курса. Студент устанавливает локальный сервер PostgreSQL, создаёт отдельную роль и базу данных для проекта, подключает Prisma ORM и проверяет соединение из TypeScript. Физические таблицы предметной области будут созданы в лабораторной работе № 6, поэтому в этой работе миграция с таблицами ещё не выполняется.

В TheraCity используется тот же общий стек PostgreSQL и Prisma ORM. StudySpace повторяет инженерную последовательность установки и конфигурации, но использует собственные имя базы, сущности и данные. Для воспроизводимости курса фиксируется Prisma ORM 6.19.2 — версия, используемая в эталонном проекте TheraCity. Переход на другую major-версию выполняется только по отдельному заданию преподавателя, поскольку структура конфигурации и способ генерации клиента могут отличаться.

2. Цель работы

Установить и проверить локальный PostgreSQL, создать изолированную роль и базу данных StudySpace, подключить Prisma ORM 6.19.2 к существующему Next.js-проекту, безопасно настроить строку подключения, сгенерировать Prisma Client и выполнить диагностический запрос к серверу базы данных.

3. Задачи работы

В ходе выполнения работы студент должен:

- установить поддерживаемую версию PostgreSQL с официальной страницы загрузки;
- сохранить сведения о выбранных компонентах, порте и каталоге данных;
- проверить запуск службы PostgreSQL;
- освоить подключение через `psql`;
- создать отдельную роль `study_app` без прав суперпользователя;
- создать базу данных `study_space` с владельцем `study_app`;
- проверить подключение под созданной ролью;
- установить одинаковые версии `prisma` и `@prisma/client`;
- инициализировать Prisma для провайдера `postgresql`;
- настроить `DATABASE_URL` через переменную окружения;
- исключить реальные секреты из системы контроля версий;

- описать generator, datasource и временную модель проверки;
- проверить схему командами prisma format и prisma validate;
- сгенерировать Prisma Client;
- создать единственный экземпляр PrismaClient для режима разработки Next.js;
- выполнить диагностический SQL-запрос из TypeScript;
- проверить npm run lint и npm run build;
- подготовить доказательства установки и соединения для отчёта.

Теоретическая часть

4.1 PostgreSQL как сервер реляционной базы данных

PostgreSQL — объектно-реляционная система управления базами данных с открытым исходным кодом. Серверный процесс принимает подключения клиентских приложений, управляет файлами данных, выполняет SQL-команды и обеспечивает согласованность данных. Один запущенный сервер PostgreSQL может обслуживать несколько баз данных. Официальное введение рекомендует использовать отдельную базу для отдельного проекта или пользователя. ([PostgreSQL: Architectural Fundamentals](#); [Creating a Database](#))

В этой работе необходимо различать:

- сервер PostgreSQL — запущенная служба, которая слушает сетевой порт;
- кластер баз данных — набор баз, которыми управляет один экземпляр сервера;
- база данных study_space — логическое пространство данных учебного проекта;
- схема public — пространство имён объектов внутри базы;
- роль study_app — учётная запись PostgreSQL, от имени которой подключается приложение;
- клиент psql — терминальная программа для отправки SQL-команд серверу.

Наличие установленного pgAdmin не заменяет сервер PostgreSQL. pgAdmin является графическим клиентом, а хранение и обработку данных выполняет сервер.

4.2 Установка PostgreSQL в Windows

Официальная страница PostgreSQL для Windows направляет пользователя к интерактивному установщику EDB. В состав установщика входят сервер PostgreSQL, pgAdmin и StackBuilder. Для лабораторной работы обязательны сервер и командные инструменты; pgAdmin можно установить как дополнительный клиент. ([PostgreSQL: Windows installers](#))

Во время установки задаются каталог программы, каталог данных, пароль административной роли postgres, порт и локаль. Обычно используется порт 5432, но документация psql указывает, что фактический порт может быть другим и задаётся параметром -p. Выбранные значения необходимо записать: без них невозможно однозначно диагностировать ошибку подключения.

Пароль роли postgres является локальным административным секретом. Его нельзя помещать в отчёт, исходный код или снимок экрана. В отчёте пароль закрывается или не попадает в кадр.

4.3 Роли, владельцы и принцип минимальных привилегий

PostgreSQL использует роли для управления доступом. CREATE ROLE создаёт роль, а атрибут LOGIN разрешает ей подключаться к серверу. Роль может быть владельцем базы и объектов. ([PostgreSQL: CREATE ROLE](#))

Next.js-приложение не должно подключаться как postgres: административная роль имеет избыточные права. Для StudySpace создаётся отдельная роль study_app, которой принадлежит только учебная база. Такой подход уменьшает область последствий ошибки приложения и делает конфигурацию проекта явной.

В лабораторной работе роль study_app не получает SUPERUSER, CREATEDB и CREATEROLE. База study_space создаётся владельцем study_app. Это учебная реализация принципа минимально необходимых прав.

4.4 База данных и схема

Команда CREATE DATABASE создаёт новую базу. Официальная документация отмечает, что имя базы должно быть уникальным внутри сервера, а выполнение требует соответствующего права. ([PostgreSQL: CREATE DATABASE](#); [Creating a Database](#))

Строка подключения будет содержать имя базы study_space и параметр schema=public. База и схема — не одно и то же. Клиент сначала подключается к конкретной базе, а затем обращается к объектам схемы. В лабораторной работе используется стандартная схема public внутри отдельной базы.

4.5 Терминальный клиент psql

psql — терминальный клиент PostgreSQL. Он принимает SQL интерактивно, из файла или через аргумент -с. Для соединения используются параметры -h для хоста, -p для порта, -U для роли и -d для базы. (PostgreSQL: psql)

```
# Подключаемся к локальной базе под прикладной ролью.  
psql -h localhost -p 5432 -U study_app -d study_space
```

Внутри psql полезны метакоманды:

- \conninfo — показывает текущее соединение;
- \l — выводит базы данных;
- \du — выводит роли;
- \dn — выводит схемы;
- \dt — выводит таблицы текущей схемы;
- \q — завершает сеанс.

Метакоманды обрабатывает `psql`, поэтому они не являются SQL-командами и не заканчиваются точкой с запятой.

4.6 Что такое ORM и из каких частей состоит Prisma ORM

ORM связывает объекты прикладного кода с записями реляционной базы. Prisma ORM предоставляет схему данных, миграции и типобезопасный клиент для Node.js и TypeScript. Официальная документация описывает Prisma ORM как набор средств с моделью данных, автоматизированными миграциями, типобезопасностью и автодополнением. ([Prisma ORM: Overview](#))

В учебном проекте используются следующие части:

- Prisma Schema — файл `schema.prisma` с источником данных, генератором и моделями;
- Prisma CLI — команды `init`, `format`, `validate`, `generate`, `migrate` и `studio`;
- Prisma Client — сгенерированный TypeScript-клиент для запросов;
- Prisma Migrate — история версионизируемых изменений физической схемы;
- Prisma Studio — графический интерфейс просмотра и редактирования данных разработки.

Prisma не заменяет PostgreSQL. PostgreSQL хранит данные и выполняет запросы, а Prisma формирует запросы и типы на основе описанной модели.

4.7 Структура `schema.prisma`

Официальная документация Prisma выделяет три части схемы: `datasource`, `generator` и `data model`. `datasource` задаёт тип источника данных, `generator` — создаваемый клиент, а блоки `model` описывают прикладные сущности. ([Prisma: Overview on Prisma Schema](#))

```
// Указывает генератор Prisma Client, совместимый с Prisma ORM 6.
generator client {
  provider = "prisma-client-js"
}

// Подключает локальную базу PostgreSQL через переменную окружения.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

В пятой работе добавляется временная модель `SetupCheck` только для проверки генерации типов. Команда `migrate` не выполняется, поэтому физическая таблица из неё не создаётся. В шестой работе временная модель будет удалена и заменена моделями предметной области.

4.8 Строка подключения PostgreSQL

Prisma использует URL подключения. Для Prisma ORM 6 URL задаётся в datasource через `env("DATABASE_URL")`. Официальный формат PostgreSQL включает пользователя, пароль, хост, порт и имя базы. ([Prisma: Connection URLs](#))

```
postgresql://USER:PASSWORD@HOST:PORT/DATABASE?schema=SCHEMA
```

Для локального проекта:

```
DATABASE_URL="postgresql://study_app:CHANGE_ME@localhost:5432/study_space?schema=public"
```

Специальные символы в имени пользователя или пароле должны быть percent-encoded. Реальное значение DATABASE_URL хранится только в локальном .env. В репозиторий помещается .env.example с заполнителями.

4.9 Переменные окружения и граница сервера

Next.js загружает переменные окружения из файлов .env*. Переменная без префикса NEXT_PUBLIC доступна серверному коду, но не встраивается в клиентский JavaScript. Официальная документация Next.js также указывает, что шаблон create-next-app добавляет .env-файлы в .gitignore. ([Next.js: Environment Variables](#))

DATABASE_URL не должна иметь префикс NEXT_PUBLIC. Она используется Prisma CLI и серверным кодом. Нельзя импортировать PrismaClient в файл с директивой "use client", иначе серверная зависимость и секретная конфигурация окажутся на неправильной стороне границы приложения.

4.10 Генерация Prisma Client

Команда prisma generate читает generator и модели schema.prisma и создаёт клиент. Типы методов клиента выводятся из моделей, поэтому после изменения схемы клиент следует сгенерировать повторно. ([Prisma CLI: Reference](#); [Prisma Schema Overview](#))

Пакеты prisma и @prisma/client должны иметь одну версию. В курсе используется 6.19.2:

```
# Устанавливаем CLI только для разработки.
npm install -D prisma@6.19.2 tsx
```

```
# Устанавливаем клиент и загрузку .env для отдельных TypeScript-скриптов.
npm install @prisma/client@6.19.2 dotenv
```

Фиксация версии является методическим решением курса. Она не означает, что более новые версии Prisma не существуют; их миграционные руководства необходимо изучать отдельно.

4.11 PrismaClient и горячая перезагрузка Next.js

Prisma рекомендует избегать создания множества экземпляров PrismaClient при горячей перезагрузке Next.js. В режиме разработки модуль может загружаться повторно, поэтому экземпляр сохраняют в globalThis. В production создаётся обычный экземпляр. ([Prisma: Next.js troubleshooting](#))

```
// Переиспользуем клиент между горячими перезагрузками development-сервера.  
const globalForPrisma = globalThis as unknown as {  
  prisma: PrismaClient | undefined;  
};
```

Один экземпляр клиента не означает одно SQL-соединение навсегда: Prisma управляет соединениями внутри клиента. Важное требование — не создавать новый `PrismaClient` при каждом запросе.

4.12 Миграции и граница пятой работы

Prisma Migrate создаёт и применяет версионизируемые SQL-миграции. Документация Prisma различает `migrate dev` для разработки и `migrate deploy` для применения уже созданных миграций в `production`. `migrate dev` не должен использоваться как `production`-команда. ([Prisma: Development and production](#))

В лабораторной работе № 5 миграция не создаётся: физические таблицы проектируются в лабораторной работе № 6. На этом этапе проверяются только установка, конфигурация, генерация клиента и соединение. Команды `db push`, `migrate dev`, `migrate reset` и ручное создание таблиц выполнять нельзя.

5. Практическое задание

Этап 1. Подготовка проекта

Продолжайте собственный StudySpace из лабораторной работы № 4. Создайте отдельную ветку или резервную копию. Проверьте:

- приложение запускается;
- `npm run lint` завершается без ошибок;
- `npm run build` завершается без ошибок;
- в проекте нет реального `.env` в истории Git;
- установленные версии Node.js и npm зафиксированы в отчёте.

Этап 2. Установка PostgreSQL

Откройте официальную страницу <https://www.postgresql.org/download/windows/> и установите поддерживаемую 64-битную версию PostgreSQL.

Зафиксируйте без секретов:

- версию PostgreSQL;
- каталог установки;
- каталог данных;
- выбранный порт;
- установлены ли Command Line Tools и pgAdmin;
- имя службы PostgreSQL.

Не показывайте пароль роли postgres в отчёте.

Windows installers

Interactive installer by EDB

Download the [installer](#) certified by EDB for all supported Postgres versions.

Note! This installer is hosted by EDB and not on the PostgreSQL community servers. If you have issues with the website it's hosted on, please contact webmaster@enterprisedb.com.

This installer includes:

- The PostgreSQL server
- pgAdmin, a graphical tool for managing and developing your databases
- StackBuilder, a package manager for downloading and installing additional PostgreSQL tools and drivers. StackBuilder includes management, integration, migration, replication, geospatial, connectors and other tools.

This installer can run in graphical or silent install modes.

The Windows installer is designed to be a straightforward, fast way to get up and running with Postgres. In the unlikely event that you have issues with the installer, please report them on the [installer Github page](#). Issues with the installed packages should be reported to the appropriate project directly.

Advanced users can also download a [zip archive](#) of the binaries, without the installer. This download is intended for users who wish to include Postgres as part of another application installer.

Platform support

The installers are tested by EDB on the following platforms. They can generally be expected to run on other comparable versions, for example, desktop releases of Windows:

Postgres Version	64 Bit Windows Platforms
18	2025, 2022
17	2022, 2019
16	2022, 2019
15	2019, 2016
14	2019, 2016
13	2019, 2016

Этап 3. Проверка сервера и psql

Откройте новый PowerShell после установки и выполните:

```
# Проверяем доступность клиентской программы.  
psql --version
```

```
# Подключаемся к служебной базе под административной ролью.  
psql -h localhost -p 5432 -U postgres -d postgres
```

Если psql не найден, используйте SQL Shell (psql) из меню Пуск или добавьте каталог bin установленного PostgreSQL в пользовательский PATH. Не копируйте путь другой версии без проверки собственного каталога.

В psql выполните `SELECT version();` и `\conninfo`, затем завершите сеанс командой `\q`.

```
Терминал - psql
Microsoft Windows [Version 10.0.22631.3593]
(с) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Student>psql -U study_app -d study_space
psql (16.1)
Введите "help", чтобы получить справку.

study_space=> SELECT current_database(), current_user, inet_server_port();

 current_database | current_user | inet_server_port
-----+-----+-----
 study_space      | study_app    |          5432
(1 row)

study_space=> █
```

Этап 4. Создание роли и базы проекта

Снова подключитесь к базе postgres под ролью postgres и выполните команды из приложения. Замените пароль на собственный локальный пароль, который не используется в других системах.

```
-- Создаём отдельную роль, от имени которой будет работать StudySpace.
CREATE ROLE study_app
WITH LOGIN
PASSWORD 'CHANGE_ME';

-- Создаём отдельную базу и назначаем прикладную роль владельцем.
CREATE DATABASE study_space
WITH OWNER = study_app
ENCODING = 'UTF8';
```

Повторный запуск CREATE ROLE или CREATE DATABASE приведёт к ошибке существующего объекта. Не удаляйте чужие роли или базы; сначала проверьте \du и \l.

Этап 5. Проверка прикладного подключения

Подключитесь уже не как postgres, а как study_app:

```
# Проверяем, что прикладная роль входит именно в учебную базу.
psql -h localhost -p 5432 -U study_app -d study_space
```

Выполните:

```
-- Показываем фактическую базу, роль и серверный порт.
SELECT current_database(), current_user, inet_server_port();
```

Результат должен содержать study_space и study_app. Команда \dt на этом этапе сообщает, что отношений нет: таблицы появятся в следующей работе.

Этап 6. Установка Prisma ORM

В корне Next.js-проекта выполните:

```
# Устанавливаем согласованные версии CLI, клиента и запуска TypeScript-скриптов.
npm install -D prisma@6.19.2 tsx
npm install @prisma/client@6.19.2 dotenv

# Проверяем фактически установленные версии.
npx prisma --version
```

В отчёте версии prisma и @prisma/client должны совпадать.

Этап 7. Инициализация Prisma

Выполните:

```
# Создаём каталог prisma, schema.prisma и шаблон переменной подключения.
npx prisma init --datasource-provider postgresql
```

Проверьте появление prisma/schema.prisma и .env. Если эти файлы уже существуют, не запускайте init повторно и не перезаписывайте их: сравните содержимое с приложением.

Этап 8. Настройка DATABASE_URL

В .env укажите собственный пароль роли study_app:

```
# Хранит локальное подключение к базе; реальный пароль не публикуется.
DATABASE_URL="postgresql://study_app:CHANGE_ME@localhost:5432/study_space?schema=public"
```

Создайте .env.example с тем же ключом и заполнителем. Проверьте .gitignore. Выполните git status и убедитесь, что .env не предлагается к коммиту.

Этап 9. Настройка schema.prisma

Добавьте generator, datasource и временную модель SetupCheck из приложения. Эта модель нужна только для генерации типизированного клиента. Не запускайте миграцию.

Выполните:

```
# Форматируем и проверяем синтаксис Prisma Schema.
npx prisma format
npx prisma validate

# Создаём типизированный Prisma Client.
npx prisma generate
```

Этап 10. Единственный экземпляр PrismaClient

Создайте `app/lib/prisma.ts` по приложению. Файл относится только к серверной части и не должен содержать директиву `"use client"`.

Этап 11. Диагностический запрос из TypeScript

Создайте `scripts/check-prisma.ts` и выполните:

```
# Запускаем серверный TypeScript-скрипт с реальным подключением.
npx tsx scripts/check-prisma.ts
```

Скрипт должен вывести `study_space` и `study_app`, а затем корректно вызвать `prisma.$disconnect()`. Если появляется P1000, проверьте имя роли и пароль. При P1001 проверьте службу, хост и порт. Не публикуйте полный `DATABASE_URL` в сообщении об ошибке.

Этап 12. Итоговая проверка

Выполните:

```
# Проверяем конфигурацию и приложение после подключения Prisma.
npx prisma validate
npx prisma generate
npm run lint
npm run build
```

```

Windows PowerShell
PS D:\study\StudySpace> npx tsx scripts/check-prisma.ts
Database: study_space
User: study_app
Port: 5432
Prisma connection check passed
PS D:\study\StudySpace>
PS D:\study\StudySpace> npm run lint
> studyspace@0.1.0 lint
> next lint

✓ No ESLint warnings or errors
PS D:\study\StudySpace>
PS D:\study\StudySpace> npm run build
> studyspace@0.1.0 build
> next build

▲ Next.js 14.2.3
- Environments: .env.production
✓ Создание оптимизированной production-сборки...
✓ Скомпилировано успешно
✓ Линтинг и проверка валидности типов...
✓ Сбор данных страниц...
  Собиратель статических страниц (0/12) ...
  Собиратель статических страниц (6/12) ...
  Собиратель статических страниц (12/12) ...
✓ Генерация статических страниц (12/12)
✓ Финализация оптимизации страниц...

Route (app)                                Size                First Load JS
o /                                           2.15 kB             98.1 kB
o /courses                                 3.42 kB             101 kB
o /courses/[id]                             2.31 kB             991 kB
o /dashboard                               2.08 kB             97.6 kB
o /profile                                 1.48 kB             96.2 kB
o /settings                                1.31 kB             95.8 kB
o /search                                  2.61 kB             100 kB
o /about                                  1.01 kB             94.7 kB
+ First Load JS shared by all              89.3 kB
  ├ chunks/abc-123def456.js                 42.1 kB
  └ other shared chunks (total)              47.2 kB

o (Static)      автоматически статически сгенерирован
f (Dynamic)     динамический рендер на сервере
o (Client)      рендер на клиенте
PS D:\study\StudySpace> |

```

Откройте проект в режиме разработки и убедитесь, что существующие страницы ЛР4 не сломаны. Таблицы и Prisma Studio с предметными данными будут проверяться в ЛР6.

Для отчёта нужны собственные скриншоты: страница выбора компонентов установщика без пароля; вывод `psql --version`; `SELECT current_database(), current_user, inet_server_port()`; вывод `npx prisma --version`; успешные `prisma validate`, `prisma generate` и `scripts/check-prisma.ts`. Если PostgreSQL уже был установлен, вместо установщика приложите список установленных компонентов и сведения о службе. Все пароли и полные строки подключения должны быть скрыты.

Не выполняйте `prisma migrate dev`, `prisma db push` или ручной `CREATE TABLE` в этой работе. Пустая база после успешной проверки соединения является ожидаемым результатом. Физическая схема создаётся в лабораторной работе № 6.

6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

- локальная служба PostgreSQL запускается;
- `psql` подключается к серверу;
- создана отдельная роль `study_app` без административных прав;
- создана база `study_space` с владельцем `study_app`;
- подключение под `study_app` подтверждено SQL-запросом;
- `prisma` и `@prisma/client` имеют версию 6.19.2;
- `prisma/schema.prisma` проходит `format` и `validate`;
- Prisma Client успешно генерируется;
- `DATABASE_URL` хранится только в локальной переменной окружения;
- `.env` исключён из Git, а `.env.example` не содержит секрета;
- `app/lib/prisma.ts` создаёт единый экземпляр клиента;
- `scripts/check-prisma.ts` подтверждает подключение;
- предметные таблицы ещё не созданы;
- `npm run lint` и `npm run build` завершаются без ошибок.

7. Содержание отчёта

Отчёт должен содержать:

- название и цель работы;
- версию Windows, Node.js, npm и PostgreSQL;
- выбранный порт и имя службы PostgreSQL;
- краткое объяснение различий сервера, базы, схемы, роли и клиента;
- SQL-команды создания роли и базы с удалённым паролем;
- скриншот `psql --version`;
- скриншот прикладного `SELECT current_database(), current_user, inet_server_port()`;
- вывод `npx prisma --version`;

- дерево новых файлов проекта;
- `schema.prisma` без секретов;
- `.env.example`;
- код `app/lib/prisma.ts`;
- код `scripts/check-prisma.ts`;
- скриншоты успешных `prisma validate` и `prisma generate`;
- скриншот успешного TypeScript-запроса;
- результаты `npm run lint` и `npm run build`;
- описание одной возникшей проблемы и способа её диагностики;
- ответы на контрольные вопросы;
- вывод по работе.

Контрольные вопросы:

- Чем сервер PostgreSQL отличается от базы данных?
- Что такое кластер баз данных PostgreSQL?
- Для чего нужна роль PostgreSQL?
- Почему приложение не должно работать под ролью `postgres`?
- Чем база данных отличается от схемы `public`?
- Какие параметры использует `psql` для подключения?
- Что показывает `\conninfo`?
- Из каких частей состоит Prisma ORM?
- Что описывают `datasource` и `generator`?
- Для чего используется `DATABASE_URL`?
- Почему `DATABASE_URL` нельзя объявлять с `NEXT_PUBLIC_`?
- Почему специальные символы пароля могут потребовать percent-encoding?
- Почему версии `prisma` и `@prisma/client` должны совпадать?
- Что делает `prisma validate`?
- Что делает `prisma generate`?
- Почему `PrismaClient` сохраняют в `globalThis` в `development`?
- Чем `migrate dev` отличается от `migrate deploy`?
- Почему в этой работе миграция не выполняется?
- Что означают типовые ошибки подключения P1000 и P1001?
- Какие данные нельзя помещать в отчёт и репозиторий?

8. Источники

Теоретическая часть подготовлена по официальной документации PostgreSQL, Prisma ORM и Next.js. Выбор имён `study_space` и `study_app`, фиксация версии 6.19.2, последовательность этапов и требования к отчёту являются методикой курса.

- PostgreSQL: Windows installers — <https://www.postgresql.org/download/windows/>
- PostgreSQL: Architectural Fundamentals — <https://www.postgresql.org/docs/current/tutorial-arch.html>

- PostgreSQL: Creating a Database — <https://www.postgresql.org/docs/current/tutorial-createdb.html>
- PostgreSQL: psql — <https://www.postgresql.org/docs/current/app-psql.html>
- PostgreSQL: CREATE ROLE — <https://www.postgresql.org/docs/current/sql-createrole.html>
- PostgreSQL: CREATE DATABASE — <https://www.postgresql.org/docs/current/sql-createdatabase.html>
- Prisma ORM v6 — <https://www.prisma.io/docs/orm/v6>
- Prisma: Overview on Prisma Schema — <https://www.prisma.io/docs/orm/v6/prisma-schema/overview>
- Prisma: Models — <https://www.prisma.io/docs/orm/v6/prisma-schema/data-model/models>
- Prisma: Connection URLs — <https://www.prisma.io/docs/orm/v6/reference/connection-urls>
- Prisma CLI Reference — <https://www.prisma.io/docs/orm/v6/reference/prisma-cli-reference>
- Prisma: Development and production — <https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>
- Prisma: Raw queries — <https://www.prisma.io/docs/orm/v6/prisma-client/using-raw-sql/raw-queries>
- Prisma: Next.js troubleshooting — <https://www.prisma.io/docs/orm/v6/more/troubleshooting/nextjs>
- Next.js: Environment Variables — <https://nextjs.org/docs/app/guides/environment-variables>

1. Структура новых и изменённых файлов

Показывает только файлы, относящиеся к подключению PostgreSQL и Prisma.

```
study-space/
├── app/
│   └── lib/
│       └── prisma.ts
├── prisma/
│   └── schema.prisma
├── scripts/
│   └── check-prisma.ts
├── .env
├── .env.example
├── .gitignore
└── package.json
```

2. SQL создания роли и базы

-- Этот фрагмент выполняется в psql под локальной административной ролью.

-- Создаём отдельную роль приложения без прав суперпользователя.

```
CREATE ROLE study_app
WITH LOGIN
PASSWORD 'CHANGE_ME';
```

-- Создаём базу проекта и назначаем прикладную роль владельцем.

```
CREATE DATABASE study_space
WITH OWNER = study_app
ENCODING = 'UTF8';
```

-- Пароль CHANGE_ME обязательно заменяется локальным секретом студента.

3. Проверка подключения через psql

Подключаемся к базе StudySpace от имени прикладной роли.

```
psql -h localhost -p 5432 -U study_app -d study_space
```

Следующие строки вводятся уже внутри psql.

```
\conninfo
SELECT current_database(), current_user, inet_server_port();
\dt
\q
```

4. Файл .env.example

Документирует формат подключения без настоящего пароля.

```
DATABASE_URL="postgresql://study_app:CHANGE_ME@localhost:5432/study_space?schema=public"
```

5. Фрагмент .gitignore

Исключаем локальные файлы окружения с секретами.

```
.env
```

```
.env.local
.env.development.local
.env.test.local
.env.production.local

# Разрешаем публиковать безопасный шаблон переменных.
!.env.example
```

6. Файл prisma/schema.prisma

```
// Генерирует Prisma Client для установленной версии Prisma ORM 6.19.2.
generator client {
  provider = "prisma-client-js"
}

// Читает локальное подключение PostgreSQL из DATABASE_URL.
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Временная модель позволяет проверить генерацию клиента.
// В ЛР6 она удаляется до создания первой миграции.
model SetupCheck {
  // Описывает тестовый первичный ключ только на уровне Prisma Schema.
  id Int @id
}
```

Файл app/lib/prisma.ts

```
// Создаёт единый серверный экземпляр PrismaClient для Next.js.

import { PrismaClient } from "@prisma/client";

// Расширяем тип globalThis полем, используемым только в development.
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

// Переиспользуем клиент после горячей перезагрузки или создаём его впервые.
export const prisma = globalForPrisma.prisma ?? new PrismaClient();

// Сохраняем экземпляр глобально только вне production.
if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

8. Файл scripts/check-prisma.ts

```
// Проверяет соединение Prisma с PostgreSQL без создания предметных таблиц.

import "dotenv/config";
import { prisma } from "../app/lib/prisma";

// Описываем форму строки, которую возвращает диагностический SELECT.
type ConnectionInfo = {
  database: string;
  role: string;
  port: number;
};

async function main() {
```

```

// Запрашиваем у сервера фактические параметры текущего соединения.
const rows = await prisma.$queryRaw<ConnectionInfo[]>`
  SELECT
    current_database() AS database,
    current_user AS role,
    inet_server_port() AS port
`;

// Выводим только безопасные диагностические сведения, без DATABASE_URL.
console.log("Prisma connected:", rows[0]);
}

// Завершаем процесс с ненулевым кодом при ошибке подключения.
main()
  .catch((error) => {
    console.error("Prisma connection failed:", error);
    process.exitCode = 1;
  })
  .finally(async () => {
    // Освобождаем ресурсы отдельного диагностического скрипта.
    await prisma.$disconnect();
  });

```

9. Скрипты package.json

```

{
  "scripts": {
    "prisma:format": "prisma format",
    "prisma:validate": "prisma validate",
    "prisma:generate": "prisma generate",
    "db:check": "tsx scripts/check-prisma.ts"
  }
}

```

В package.json комментарии запрещены синтаксисом JSON. Назначение каждой команды приведено в теоретической и практической частях.

10. Команды установки и проверки

```

# Устанавливаем согласованные версии Prisma и TypeScript-runner.
npm install -D prisma@6.19.2 tsx
npm install @prisma/client@6.19.2 dotenv

# Инициализируем Prisma только при отсутствии каталога prisma.
npx prisma init --datasource-provider postgresql

# Проверяем схему, генерируем клиент и выполняем диагностический запрос.
npx prisma format
npx prisma validate
npx prisma generate
npm run db:check

# Проверяем, что подключение Prisma не сломало Next.js-проект.
npm run lint
npm run build

```

11. Что студент должен заменить

- пароль CHANGE_ME — на собственный локальный пароль роли study_app;

- порт 5432 — на фактический порт своей установки, если он изменён;
- имена `study_app` и `study_space` — только если собственная предметная область требует других согласованных имён;
- пути файлов — на структуру собственного проекта, если она отличается;
- версии Prisma — не изменять без указания преподавателя;
- все скриншоты — на результаты собственного компьютера.

Нельзя копировать реальный пароль, полный `DATABASE_URL`, данные роли `postgres` или снимки экрана другого студента.